H2: Variabelen en datatypes

Labo 2 · week 2 · variabelen, namen en rekenen

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Opwarmen**: welk datatype?
2. **Variabelen**: declareren, een waarde geven en overschrijven
3. **Namen**: welke namen mogen, en welke zijn goed?
4. **Expressies en operatoren**: rekenen in C#
5. **Het datatype van een resultaat**: waarom 9 / 2 gelijk is aan 4



Vandaag leren we rekenen. Rustig, de computer doet het werk. Wij krijgen de schuld.

Opwarmen

Welk datatype? 🤔

Gegeven	Datatype
het aantal studenten in de klas	`int`
je gemiddelde op het examen	`double`
heb je een klantenkaart?	`bool`
de naam van je straat	`string`

😈 *Je gemiddelde als `int`? Dan word je 13,9 gewoon 13. Wreed.*

Variabelen

Een variabele maken

Een **variabele** koppelt een **naam** aan gegevens. Elke variabele heeft een **datatype**.

```
1  int age;                                  // declareren
2  string firstName, lastName, address;      // meerdere tegelijk
3  int myAge = 37;                           // meteen een beginwaarde
```

- **Declareren**: het datatype en de naam opgeven. C# reserveert geheugen.
- **Initialiseren**: de variabele meteen een **beginwaarde** geven.

Een variabele declareren zonder ze te gebruiken: zoals een parkeerplaats reserveren en dan thuisblijven.

Een waarde toekennen

Met `=` geef je een variabele een waarde. Dat heet een **toekenning**, en die werkt van **rechts naar links**.

```
1  int temperatureYesterday = 20;  // een literal
2  int temperatureToday = 25;
3  int otherAge = myAge;            // een andere variabele
```

Een **literal** is een waarde die letterlijk in je code staat: `20`, `"Tim"`, `true`.

 `=` *is geen "is gelijk aan", maar "wordt". De wiskundeleerkracht in je hoofd moet even zwijgen.*

Het type van een literal

Literal	Datatype
`125`	`int`
`12.5`	`double`
`12.5f`	`float`
`12.5m`	`decimal`
`125L`	`long`
`'q'`	`char`
`"pikachu"`	`string`
`true`	`bool`



`12.5m`: de `m` staat voor money. Of voor "mag ik nog een frietje".

Het type moet passen

```
1  string text = 4;      // ❌ CS0029
2  int number = "4";     // ❌ CS0029
```

> ⚠️ **WAARSCHUWING**
>
> Cannot implicitly convert type ‘int’ to ‘string’

`4` is een `int`, `"4"` is een `string`. Voor C# zijn dat totaal verschillende dingen.

Een getal in een tekstvariabele steken is als soep in een vergiet gieten. Het lukt niet.

Overschrijven

Een nieuwe waarde **overschrijft** de oude. De oude waarde is dan **weg**.

```
1  int temperatureYesterday = 20;
2  temperatureYesterday = 25;   // 20 is weg
```

Heb je de oude waarde nog nodig? Bewaar ze **eerst** in een andere variabele:

```
1  int temperatureDayBefore = temperatureYesterday;
2  temperatureYesterday = 25;
```



Overschrijven is definitief. Zoals je oude gsm-nummer aan je oma doorgeven.

Namen

Regels voor namen

Een naam die je zelf kiest, heet een **identifier**.

- Het **eerste teken** is een letter of een liggend streepje `_`.
- De **andere tekens**: letters, cijfers of `_`. Geen spaties, geen `+`, geen `-`.
- **Geen sleutelwoord**: `class` of `int` mag niet.
- **Hoofdlettergevoelig**: `age` en `Age` zijn twee verschillende namen.

😈 *Een variabele met een spatie in de naam? C# ziet dan twee variabelen en geen idee wat ermee te doen.*

Mag het? 🤔

Naam	Toegelaten?
`employee`	✅
`christmas2026`	✅
`my age`	❌ geen spaties
`4thPlace`	❌ mag niet met een cijfer beginnen
`_ILOVE2026`	✅ mag, maar het is geen goede naam
`Tom+Jerry`	❌ enkel letters, cijfers en `_`
`class`	❌ een sleutelwoord
`WriteLine`	✅

😈 *`Tom+Jerry`: in een tekenfilm een duo, in C# een foutmelding.*

Goede namen

Wat **mag** is niet altijd wat **goed** is. Onze afspraken:

- Een **duidelijke** naam: `weight` of `age`, niet `a` of `meuh`.
- **camelCase** voor variabelen: `numberOfStudents`, `temperatureToday`.
- **PascalCase** voor klassen en methoden: `VariabelenEnDatatypes`, `AddUp`.
- In het **Engels**, zonder afkortingen.

> **BELANGRIJK**
>
> Naamgeving die niet volgens de afspraken is, kost op een evaluatie **2 punten**.

Een variabele `x` noemen is als je kind "Kind" noemen. Technisch correct, maar niemand weet wie je bedoelt.

Expressies en operatoren

Expressies

Een **expressie** is iets dat een **waarde** oplevert: `3 + 2`, `7`, `Console.ReadLine()`.

```
1  int temperatureDifference = temperatureToday - temperatureYesterday;
2  int width = 15;
3  int height = 20 * width;
```

C# rekent eerst het **rechterdeel** uit, en bewaart het resultaat in de variabele links.



Een expressie levert altijd iets op. Meer dan sommige vergaderingen.

Operatoren en volgorde

`+` `-` `*` `/`, met dezelfde volgorde als in de wiskunde:

1. **haakjes**
2. **vermenigvuldigen en delen**: `*` en `/`
3. **optellen en aftrekken**: `+` en `-`

Expressie	Resultaat
`3 + 5 * 2`	`13`
`(3 + 5) * 2`	`16`
`10 - 2 * 3`	`4`
`(10 - 2) * 3`	`24`

Meneer Van Dalen wacht op antwoord. Ook in C#.

Rekenen met variabelen

Hoe zwaar voel je je op Mars?

```
1  double weightOnEarth = 80.3;    // kg
2  double gravityEarth = 9.81;     // m/s²
3  double gravityMars = 3.711;     // m/s²
4  double weightOnMars = (weightOnEarth / gravityEarth) * gravityMars;
5  Console.WriteLine("Op Mars voel je je alsof je "
6      + weightOnMars + " kg weegt.");
```

```
Op Mars voel je je alsof je 30,376483180428128 kg weegt.
```

Op Mars weeg je 30 kilo. Eindelijk een dieet dat werkt. Enkel de reis is wat lang.

Het datatype van een resultaat

`int` gedeeld door `int`

De datatypes in je berekening bepalen het datatype van het **resultaat**.

```
1  int number1 = 9;
2  int number2 = 2;
3  int result = number1 / number2;
4  Console.WriteLine(result);
```

```
4
```

Een `int` gedeeld door een `int` geeft een **`int`**. Alles na de komma is **weg**.

9 gedeeld door 2 is 4. Wiskundig fout, maar C# is daar heel zeker van.

Types mengen

Rekent C# met een `int` **en** een `double`, dan is het resultaat een **`double`**.

```
1  int number1 = 9;
2  double number2 = 2.0;
3  double result = number1 / number2;
4  Console.WriteLine(result);  // 4,5
```

Andersom gaat niet:

```
1  int result = 3 / 5.6;  // ❌ CS0266: een double past niet in een int
```

Een `double` in een `int` steken is als een kingsize dekbed in een eenpersoonshoes proppen.

Wat is het resultaat? 🤔

Expressie	Resultaat
`7 / 2`	`3`
`7 / 2.0`	`3,5`
`1 / 2 * 10`	`0`, want `1 / 2` is al `0`
`10 * 1 / 2`	`5`, want `10 * 1` is `10`, en `10 / 2` is `5`
`10000.0 * (1 / 2)`	`0`, eerst de haakjes: `1 / 2` is `0`
`10000.0 * (1.0 / 2)`	`5000`

😈 *"Je krijgt de helft van mijn loon." Met `int`-deling: 0 euro. Lees altijd de kleine lettertjes.*

Samengevat

Wat	Voorbeeld
declareren	`int age;`
initialiseren	`int age = 18;`
toekennen (van rechts naar links)	`age = otherAge;`
goede namen	`numberOfStudents` (camelCase, Engels)
volgorde van berekeningen	haakjes, dan `*` en `/`, dan `+` en `-`
`int` / `int`	een `int`: `9 / 2` is `4`
`int` en `double` gemengd	een `double`: `9 / 2.0` is `4,5`

Je kan nu rekenen in C#. Je rekenmachine overweegt een carrièreswitch.

Aan de slag 💻

Maak in de klasse `VariabelenEnDatatypes`:

1. **AddUp**: tel twee getallen op (als dat nog niet af is)
2. **FuelConsumption**: bereken het verbruik van een auto
3. **LittleMath**: reken enkele expressies uit
4. **Average**: het gemiddelde van drie getallen
5. **MultiplicationTables**: de tafel van 411, stap per stap
6. **Space**: je gewicht op andere planeten
7. **BicycleSeller**: de prijs van een fiets met btw

👉 Bekijk ook de **opdracht voor het project** op DigitAP.

😈 *Zeven oefeningen. Lucky seven. Of gewoon zeven.*